

Polynomials over Finite Fields: Interpolation, Identity Testing, and the NTT

Lecture 2 of 6 — Foundations

SEC 595 — Encrypted Computing

- 1 The Ring $\mathbb{F}_p[X]$
- 2 Interpolation: Coefficients $\leftrightarrow$ Evaluations
- 3 Polynomial Identity Testing
- 4 Vanishing Polynomials and Quotients
- 5 Roots of Unity in $\mathbb{F}_p$
- 6 The Number-Theoretic Transform

Where we are going

Lecture 1 built $\mathbb{Z}_q$: one modulus, one set of residues.

This lecture builds

$$\mathbb{F}_p[X], \quad p \text{ prime},$$

the ring of polynomials over a finite field, and extracts from it four tools that between them account for most of the course.

The organising idea — one duality

A polynomial of degree less than k can be written down in two completely equivalent ways:

$$\underbrace{(a_0, a_1, \dots, a_{k-1})}_{\text{coefficients}} \iff \underbrace{(a(x_1), \dots, a(x_k))}_{\text{evaluations at } k \text{ distinct points}}$$

Moving between the two representations is **interpolation**.

Almost everything in this lecture is an application of being able to move between them cheaply.

The working field

Throughout, p is prime and $\mathbb{F}_p = \mathbb{Z}_p$ is the field of Lecture 1.

Most worked examples use $\mathbb{F}_{17}$, because

$$17 - 1 = 16 = 2^4$$

makes it the smallest field in which the NTT is interesting.

$$\mathbb{F}_p[X] = \{ a_0 + a_1X + \cdots + a_dX^d : a_i \in \mathbb{F}_p \}$$

A commutative ring under the usual addition and multiplication.

We write $\deg a$ for the degree, with the convention $\deg 0 = -\infty$.

Theorem (division algorithm)

For $a, b \in \mathbb{F}_p[X]$ with $b \neq 0$ there exist *unique* $q, r \in \mathbb{F}_p[X]$ with

$$a = q \cdot b + r, \quad \deg r < \deg b.$$

Input: a and b . **Output:** q and r .

The analogy that carries everything across

Backward reference

Lecture 1's division algorithm for $\mathbb{Z}$ was $a = qn + r$ with $0 \leq r < n$.

The clause that made q, r unique there was the *size* restriction on the remainder.

Here the same job is done by $\deg r < \deg b$: degree replaces absolute value.

That single substitution carries the whole Euclidean apparatus across.

Corollary

The Euclidean algorithm computes $\gcd(a, b)$ in $\mathbb{F}_p[X]$; the extended version produces u, v with

$$ua + vb = \gcd(a, b);$$

and a is invertible modulo f exactly when $\gcd(a, f) = 1$.

Nothing new has to be learned. It is Lecture 1 with $\deg$ in place of $|\cdot|$.

Forward reference

This corollary is what lets us invert elements of $R_q = \mathbb{Z}_q[X]/(X^n + 1)$ in Lecture 3.

It is also the algorithm behind the Li–Micciancio recovery attack on CKKS: given a decryption result and the ciphertext, solve for the secret by inverting a ring element.

Theorem

$a(\alpha) = 0$ if and only if $(X - \alpha) \mid a(X)$.

Proof. Divide: $a(X) = q(X)(X - \alpha) + r$ with $\deg r < 1$, so r is a constant. Evaluating at α gives $r = a(\alpha)$. □

The key counting fact

Corollary

A nonzero polynomial of degree d over a **field** has at most d roots.

Peel off one factor $(X - \alpha)$ per root; the degree drops each time.

This corollary does more work in this course than any other single fact.

“Over a field” is essential

In $\mathbb{Z}_8$ the polynomial $X^2 - 1$ has **four** roots:

$$1, \quad 3, \quad 5, \quad 7.$$

The proof needs the absence of zero divisors, and $\mathbb{Z}_8$ has them.

This failure is not a curiosity: it is exactly what makes arithmetic in R_q subtler than arithmetic in $\mathbb{F}_p[X]$.

Agreement at k points means equality

Two polynomials of degree $< k$ that agree at k points are **equal**.

$$\deg(a - b) < k \quad \text{and} \quad a - b \text{ has } k \text{ roots} \quad \implies \quad a - b = 0.$$

Everything in the next two sections follows from that observation.

Theorem

Let $x_1, \dots, x_k \in \mathbb{F}_p$ be distinct and $y_1, \dots, y_k \in \mathbb{F}_p$ arbitrary. There is exactly one polynomial a of degree $< k$ with $a(x_i) = y_i$ for all i , namely

$$a(X) = \sum_{i=1}^k y_i \ell_i(X), \quad \ell_i(X) = \prod_{\substack{j=1 \\ j \neq i}}^k \frac{X - x_j}{x_i - x_j}.$$

Why the formula is obvious once seen

Each basis polynomial ℓ_i is

$$\ell_i(x_i) = 1, \quad \ell_i(x_j) = 0 \quad (j \neq i),$$

so the weighted sum $\sum_i y_i \ell_i$ clearly passes through every node.

Existence is that formula. **Uniqueness** is the counting fact.

Note the division: this needs a *field*, not merely a ring.

Example — Lagrange interpolation

Let $x_1 = 1$, $x_2 = 2$, $x_3 = 3$, $y_1 = 10$, $y_2 = 5$, and $y_3 = 1$ Find ℓ_1 , ℓ_2 , and ℓ_3 .

Example — reconstructing a Shamir secret in $\mathbb{F}_{17}$

A dealer picks $a(X) = 13 + 10X + 2X^2$ over $\mathbb{F}_{17}$ and hands out $a(i)$ for $i = 1, \dots, 5$:

$$(a(1), \dots, a(5)) = (8, 7, 10, 0, 11),$$

keeping the secret $S = a(0)$.

Using only the shares at nodes 1, 2, 3, recover S .

Two shares determine nothing

For *any* candidate secret s there is a unique degree-2 polynomial through the two known points and $(0, s)$.

So two shares are consistent with every secret, equally.

That sentence is the entire security argument for Shamir's scheme, and it is a statement about **degrees**, not about computational hardness.

Secret sharing is *information-theoretically* secure. Nothing else in this course is.

A second place to hide information — the roots

Section 2.1 hid a secret in a **coefficient** and revealed *evaluations*.

The dual trick hides a set in the **roots**:

$$P(X) = \prod_{s \in S} (s - X) \quad \implies \quad P(y) = 0 \iff y \in S.$$

The polynomial is public; the set is not obviously readable from it once the coefficients are encrypted.

Example — private set intersection (L16)

Alice holds $\{3, 4, 5\}$ and forms the polynomial vanishing exactly on her set. She sends its *encrypted* coefficients.

Bob holds a single y and may evaluate homomorphically. What should he return so that Alice's decryption reveals y when y is in her set, and nothing at all otherwise?

The problem

We want to check whether two polynomials A and B of degree $< d$ are equal.

They are enormous — say $d = 2^{20}$ — and we want to spend $O(1)$ work.

Comparing coefficients costs d . Instead: evaluate both at a **single random point**.

Theorem (univariate case)

Let $P \in \mathbb{F}[X]$ be nonzero of degree $\leq d$, and let $S \subseteq \mathbb{F}$ be finite. Then

$$\Pr_{r \leftarrow S} [P(r) = 0] \leq \frac{d}{|S|}.$$

Proof. P has at most d roots, and r is uniform on S . □

A one-line proof of the fact that underwrites every modern proof system.

The multivariate version

Real proof systems use polynomials in many variables.

The statement is unchanged; d becomes the **total degree**:

$$\Pr_{r \leftarrow S^m} [P(r) = 0] \leq \frac{d}{|S|}.$$

We will only need the univariate case.

The whole idea of a SNARK, in one line

Corollary

To check $A = B$, sample r uniformly and check $A(r) = B(r)$.

If $A \neq B$ this fails to detect the difference with probability at most $d/|\mathbb{F}|$.

One field element stands in for the whole comparison.

Example — soundness error at SNARK scale

Take $d = 2^{20}$ and $|\mathbb{F}| \approx 2^{254}$, a typical SNARK field.

Bound the probability that a single random evaluation fails to catch $A \neq B$. How many coefficient comparisons has that one check replaced?

The vanishing polynomial of a set

A vanishing polynomial for a set of points H is the unique polynomial $Z_H(X)$ that evaluates to zero at every single point in H , and is non-zero everywhere else.

Let $H \subseteq \mathbb{F}_p^\times$ be a subgroup of order N — concretely, the set of N -th roots of unity.

Definition

The **vanishing polynomial** of H is

$$Z_H(X) = \prod_{h \in H} (X - h).$$

Proposition

If H is the group of N -th roots of unity then

$$Z_H(X) = X^N - 1.$$

Proof. Every $h \in H$ satisfies $h^N = 1$, so each is a root of $X^N - 1$. There are N of them, all distinct, and $X^N - 1$ is monic of degree N ; by the counting fact there are no others. □

Why that is a remarkable convenience

The vanishing polynomial of an arbitrary set of size N has $N + 1$ coefficients, and evaluating it costs N multiplications.

On a multiplicative subgroup it has **two**, and evaluating it is one exponentiation:

$$Z_H(r) = r^N - 1.$$

The verifier's work does not grow with the size of the computation.

Theorem

P vanishes on all of H if and only if $Z_H \mid P$ — that is, if and only if there is a quotient Q with

$$P(X) = Q(X) \cdot Z_H(X).$$

A statement about N separate conditions has become a single divisibility.

Example — the vanishing polynomial over $\mathbb{F}_{17}$

Take $N = 4$ and $\omega = 13$, which has order 4 modulo 17.

Write down H and Z_H , and check the answer against the sum and the product of the elements of H .

The arithmetisation pattern

The shape of every modern proof system:

- a computation is encoded as a constraint that must hold at every step;
- the steps are indexed by H ;
- “the constraint holds at every step” becomes “the constraint polynomial C vanishes on H ”, which becomes

$$C(X) = Q(X) \cdot Z_H(X).$$

The prover supplies Q ; the verifier picks random r and checks $C(r) = Q(r)(r^N - 1)$.

A claim about a long computation has become one field-element identity.

When does a subgroup of the right size exist?

Proposition

$\mathbb{F}_p^\times$ is cyclic of order $p - 1$, and contains an element of order N if and only if $N \mid p - 1$.

Such an element is a **primitive N -th root of unity**, written ω .

Sections 4 and 6 both need a subgroup of a prescribed size. This is the whole availability condition, and it is Lecture 1's.

Example — roots of unity in $\mathbb{F}_{17}$

It is a fact that 3 is a primitive root of $\mathbb{F}_{17}$.

Confirm it, and then write down primitive roots of unity of orders 16, 8, 4 and 2.

Definition

The **two-adicity** of p is the largest v with $2^v \mid p - 1$.

It is the largest power-of-two NTT the field supports.

One integer, computed from q alone, that decides the arithmetic of a whole standard.

The fields we will meet

Field	$p - 1$	2-adicity	Used by
$p = 17$	2^4	4	our examples
$p = 3329$	$2^8 \cdot 13$	8	ML-KEM
$p = 8380417$	$2^{13} \cdot 1023$	13	ML-DSA
$p = 2^{64} - 2^{32} + 1$	$2^{32}(2^{32} - 1)$	32	STARK provers

The last is chosen *for* its two-adicity; the first three were chosen for other reasons and their two-adicity followed.

Forward reference

Both ML-KEM and ML-DSA use ring dimension $n = 256$. Splitting $\mathbb{Z}_q[X]/(X^n + 1)$ all the way into linear factors needs a primitive $2n = 512$ -th root of unity, hence $512 \mid q - 1$.

ML-DSA has two-adicity 13, so it does.

ML-KEM has two-adicity 8: $256 \mid q - 1$ but $512 \nmid q - 1$. Its transform stops one level early, at 128 quadratic factors — that is what *incomplete NTT* means.

The negacyclic construction is built properly in Lecture 3.

The transform is evaluation on H

Fix $N \mid p - 1$ and a primitive N -th root of unity ω . For $a = (a_0, \dots, a_{N-1})$,

$$\text{NTT}(a)_j = \hat{a}_j = a(\omega^j) = \sum_{i=0}^{N-1} a_i \omega^{ij}, \quad j = 0, \dots, N - 1.$$

Coefficients in, evaluations out — the left-to-right direction of the duality.

The inverse is interpolation, in closed form

$$\text{NTT}^{-1}(\hat{a})_i = a_i = N^{-1} \sum_{j=0}^{N-1} \hat{a}_j \omega^{-ij}.$$

The same transform with ω^{-1} in place of ω , scaled by N^{-1} .

Both N^{-1} and ω^{-1} exist in $\mathbb{F}_p$: $N \mid p-1$ so $p \nmid N$, and ω is a unit.

The NTT is not a new idea

It is Lagrange interpolation from Section 2, on the special node set

$$H = \{\omega^0, \omega^1, \dots, \omega^{N-1}\}.$$

What the special node set buys is **speed**: the naive formula costs $O(N^2)$, and the structure of H reduces it to $O(N \log N)$.

Separate the even- and odd-indexed coefficients:

$$a(X) = a_e(X^2) + X \cdot a_o(X^2).$$

Since ω^2 has order $N/2$, evaluating a_e and a_o on the subgroup of order $N/2$ is **two transforms of half the size**.

Nothing is being approximated: this is an identity.

The recursion and its cost

Recombining the two half-size transforms costs $O(N)$, so

$$T(N) = 2 T(N/2) + O(N) \quad \implies \quad T(N) = O(N \log N).$$

$\log_2 N$ layers, each of N operations.

For $N = 256$: 2048 operations instead of 65,536.

The butterfly

Write $E_j = a_e(\omega^{2j})$ and $O_j = a_o(\omega^{2j})$ for $j = 0, \dots, \frac{N}{2} - 1$.

$$\begin{aligned}(a_0, a_2, a_4, \dots) &\longrightarrow \text{NTT}_{N/2} \longrightarrow (E_0, \dots, E_{N/2-1}) \\(a_1, a_3, a_5, \dots) &\longrightarrow \text{NTT}_{N/2} \longrightarrow (O_0, \dots, O_{N/2-1})\end{aligned}$$

$$\hat{a}_j = E_j + \omega^j O_j, \quad \hat{a}_{j+N/2} = E_j - \omega^j O_j.$$

The minus sign is free: $\omega^{j+N/2} = -\omega^j$. One multiplication serves two outputs.

Example — a 4-point transform, directly

Work in $\mathbb{F}_{17}$ with $N = 4$ and $\omega = 13$, so that

$$(\omega^0, \omega^1, \omega^2, \omega^3) = (1, 13, 16, 4).$$

Take $a(X) = 1 + 2X + 3X^2 + 4X^3$ and compute $\text{NTT}(a)$ straight from the definition.

Example — the same transform by Cooley–Tukey

Same a , same $\omega = 13$.

Split a into a_e and a_o , transform each on $\{\omega^0, \omega^2\}$, and recombine with the butterfly.

Check the answer against the previous slide, and count the multiplications each route used.

Example — inverting the transform

Still in $\mathbb{F}_{17}$ with $N = 4$, $\omega = 13$.

Find N^{-1} and ω^{-1} , then recover a_0 from $\hat{a} = \text{NTT}(a)$ using the inverse formula.

Why we care — evaluation is a ring homomorphism

Multiplying two polynomials directly costs $O(N^2)$ coefficient multiplications.

In the evaluation representation it costs N , because

$$(a \cdot b)(\omega^j) = a(\omega^j) \cdot b(\omega^j).$$

Convolution in one representation is pointwise multiplication in the other.

$$a \cdot b = \text{NTT}^{-1}(\text{NTT}(a) \odot \text{NTT}(b))$$

$$\underbrace{O(N \log N)}_{\text{two transforms}} + \underbrace{O(N)}_{\text{pointwise}} + \underbrace{O(N \log N)}_{\text{one inverse}} \quad \text{instead of} \quad O(N^2).$$

This is the single most important algorithm in the course.

Forward reference

Two adjustments are needed before this is usable in Lecture 3.

First: the product of two degree- $< N$ polynomials has degree up to $2N - 2$, so a length- N transform computes the product modulo $X^N - 1$ — a *cyclic* convolution.

Second: R_q needs multiplication modulo $X^n + 1$, the *negacyclic* convolution. The fix is a twist by a primitive $2n$ -th root ψ with $\psi^2 = \omega$: scale $a_i \mapsto \psi^i a_i$ before the transform and undo it afterwards.

That is why the standards need $2n \mid q - 1$ rather than $n \mid q - 1$.

- 1 $\mathbb{F}_p[X]$ has a division algorithm, hence gcds, Bézout and inverses — Lecture 1 transplanted from $\mathbb{Z}$ to polynomials.
- 2 A nonzero polynomial of degree d over a *field* has at most d roots. Nearly every argument in this lecture is a corollary.
- 3 Coefficients and evaluations at k distinct points are two representations of the same degree- $< k$ polynomial; Lagrange moves between them.

- 4 Hiding data in the coefficients gives secret sharing; hiding it in the roots gives private set intersection.
- 5 Schwartz–Zippel makes a random evaluation a sound test of polynomial identity, with error $d/|\mathbb{F}|$.
- 6 On a multiplicative subgroup H of order N the vanishing polynomial is just $X^N - 1$, and “holds everywhere on H ” becomes “divisible by $X^N - 1$ ”.

- 7 $\mathbb{F}_p$ has an N -th root of unity if and only if $N \mid p - 1$. The two-adicity of p decides how far a power-of-two NTT can go.
- 8 The NTT is interpolation on the roots of unity, computed in $O(N \log N)$. It turns polynomial multiplication into pointwise multiplication.

Lecture 3 — The Ring R_q

Structure and fast arithmetic: quotient rings, $X^n + 1$, negacyclic convolution, the expansion factor, and the negacyclic NTT.

Lecture 1 and Lecture 2, put together into the object every scheme in the course computes in.